$2a + 3 + b = 0$

$-2a - 3 + b = 0$

$a = \frac{-3}{2}$, $b = 0$

$(2a - b) = -3$

59

$x_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix} + x_2 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

$0 \quad 0 \rightarrow 0$

$1 \quad 0 \rightarrow 0$

$0 \quad 1 \rightarrow 0$

$1 \quad 1 \rightarrow 1$

33A, 14NA, 18D

Web3, Blockchain

(12/1/25)

Smart Contracts, Ethereum (automatically executing computer programs on a blockchain)

DAOs

PoW → BTC, Litecoin (LTC), etc.

PoS → ETH(2.0), Cardano, Solana, etc.

decentralized ledger → records transaction using cryptography.

(record book)

shared database → accessible to all participants in the network.

process of obscuring data using mathematical algorithms.

=

Gas (fuel for smart contract execution)

Gas → computational cost required to execute operations on a blockchain.

(eg. adding nos, storing data, etc)

measured in. Gwei

1 Gwei = 10^{-9} ETH

21,000 gas : Typical gas cost for simple ETH transfer

Gas limit → max. gas a user is willing to pay for a transaction. (operation)

Gas price → amt. a user is willing to pay per unit of gas.

Block gas limit → max. gas all transactions in a block can consume. (collectively)

(varies bet. blocks)

determined by the network, adjustable, ensures no block become too large and affect network's performance.

Txn hash (receipt)

(unique)

serves as proof that a txn was validated and added to blockchain.

Higher gas prices → miners/validators are incentivized to prioritize that user's transaction.

More users competing for block space → higher gas price

Total fee = Gas Used × Gas price

(after) → Transaction fee (Pre-EIP-1559)

EIP-1559 → introduced new fee model.

(Ethereum Improvement Proposal)

Txn fee (Transaction) = Gas Used × (Base fee + Tip)

Base fee

fixed min. fee

burned post transaction. (removed from circulation)

dynamically adjusted based on network demand.

(doesn't go to miners/validators)

Priority fee

(optional) additional fee to incentivize miners/validators to prioritize one's transaction.

(per unit gas)

Nonce → Number only used once → (PoW) random no. miners adjust to solve the cryptographic puzzle.

(Minting → creating new tokens or coins.)

(PoS) Txn nonce → sequential no. incremented with every txn from an ethereum account.

Oracle → services or systems that connects blockchain to external data sources, provide external computations, etc.
(e.g. Chainlink)
(e.g. weather updates)

Genesis block → 1st block in a blockchain.

holds ETH and interacts with smart contracts.)

- unique identity on the ethereum network.
- Public address
- private key → used to "sign" txns.
- managed by a wallet.
  (e.g. MetaMask)

(18/1/25)

Hash → a unique fixed length string
- identifies a piece of data.
- created by placing data into a hash fn.

(e.g. SHA-256, Keccak-256, etc.) (cryptographic hash fn's)

(e.g. Division method, multiplicative (modular) hashing, etc.)
(Normal hash fn's)

(e.g.) New acc.: Nonce: 0
Txn 1 → Nonce += 1
Txn 2 → Nonce += 2
So, current nonce = 2

Elliptic Curve digital signature algorithm.
(generates private, public key pair.)

Coinbase txn → 1st txn in a block → created by miners/validators to claim block reward

(19/1/25)

Consensus → mechanism used to agree on the state of a blockchain.
(roughly)
- Chain selection → longest chain rule
- Sybil resistance → PoW, PoS

(some) Attacks → * Sybil attack → Attacker creating a lot of accounts on the blockchain network to gain/ majority.

* 51% Attack
When a single entity/group controls > 50% of ~~network's~~ the network's computational power or ~~state~~ stake.
(PoW) (PoS)
(Controlling > 50% of hash rate) (Controlling > 50% of total staked tokens)

Layer 1 → base layer (data, settlement layer)
(e.g. chainlinks, arbitrum, optimism, etc.)
(e.g. Btc, Eth, Avalanche, etc.) blockchain implementation.

Layer 2 → any application added on top of a layer 1 / blockchain.

Blockchain Trilemma
A blockchain can only achieve 2 out of 3 properties.
- Decentralization
- Security
- Scalability

(solved by) → Rollups (Bundle of txns aggregated off-chain)
- L2 Rollup
- Storing soln's
- L1 Blockchain

Bundle of transactions submitted on-chain (L1)

Where do slashed ~~cat~~ dems go? → burnt

$12 + 5 \times 12 + 1 \times 12^2 + 5 \times 12^3 = 10272$

$2 \times 12^3 + 7 \times 12^2 + 8 \times 12 + 5 = 4565$

12 | 5707
12 | 475 → 2
12 | 39 → 7
3 → 3

L2 scaling soln's (types)

Rollups → Optimistic rollups → operate under assumption that all txns are valid unless proven otherwise.

→ Zero-knowledge rollups → uses cryptographic proofs to ensure txn. validity.

Stage → 0 (centralized rollups)
→ 1 (decentralized sequencers)
→ 2 (fully decentralized rollups)

maturity level in terms of decentralization, security and integration with the base blockchain.

Bridging → "moving" funds (ETH e.g.) from one chain to another. (ETH mainnet) (zksync mainnet)

* locking and unlocking method
* minting and burning bridge.

Solidity → contract oriented, statically typed
(Software Package Data Exchange)

(No print statement)
(No floating point no.s)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract Greet {
    string public greet = "Hello, World!";
}
```

EVM → decentralized computing environment that executes smart contracts on the Ethereum network.

Write & ~~Read~~ Read →
- * Stack
- * Memory
- * Storage
- * Transient storage
- * Calldata
- * Code
- * Returndata

Write only → * Logs

Read only → * Txn data (& Blobhash)
- * Chain data
- * Gas data
- * Program counter
- * ~~Others~~ etc.

ABI → Application Binary Interface
- serves as "blueprint" for interacting with a contract.
- (Data interchange format) JSON formatted description of a contract's fn's, events, other details.

Mapping (string ⇒ uint256) public myMapping;
(keyword) [key] [value] (visibility) (name)
(similar to Python dictionary)
myMapping['_'] = _;

(21/1/25)
globally available variables (block & txn properties)
- msg.value → no. of wei sent with the message.

10^{18} wei = 1 Eth

Interface → defines a contract's fn signatures without providing their implementations.

(eg)
```
interface IExample {
    function getValue() external view
        returns (uint256);
    function setValue(uint256 _value) external;
}
```

Chainlink
- ~~data~~ decentralized oracle network.
- enables integration of external data and computation into a smart contract.
  - hybrid smart contract.

(e.g. ~~features~~ features)
- * Data feeds
- * Verifiable Random no. Generator
- * Automation (previously ~~known as~~ "keepers")
- * Functions

Library (similar to utility classes in other programming languages)
- reusable pieces of code that can be deployed once and then shared across multiple contracts.

(22/1/25)

Ways to send funds from a contract

{the contract itself}

- transfer → payable(msg.sender).transfer(address(this).balance);
- send → bool sendSuccess = payable(msg.sender).send(address(this).balance);
  require(sendSuccess, "send failed");
- call (low level) (Recommended)
  (bool callSuccess, ~~bytes memory dataReturned~~) (returns 2 values)
  = payable(msg.sender).call{value: address(this).balance}("");
  require(callSuccess, "call failed");

Modifiers → allow embedding custom code within any fn to modify its behaviour.

function withdraw(uint amt) public onlyOwner {...}

```
modifier onlyOwner {
    require(msg.sender == owner, "Not the owner");
    _;
}
```

(placeholder for the rest of the modified fn's code)

IPFS (Decentralized InterPlanetary FileSystem) protocol that allows users to store files and access files, websites and apps without a central server.

Immutable (set at deployed time)

constant (set at compiled time)

Special fns

- receive() → gets called when sent txn without any other data. (calldata)
- fallback() → gets called when sent txn with calldata that doesn't refer to any valid fn.
- constructor() → gets called automatically at deployment.

RPC → Remote Procedure Call → comms protocol that allows interactions b/w apps & blockchain nodes.

(25/1/25)

Foundry → smart contract development framework.

forge init
forge build
anvil (--block-time)

import "forge-std/Script.sol"

forge script ... --rpc-url ... --broadcast --private-key

$cast --to-base __ dec
(hex to decimal)

$cast wallet import keyName --interactive
(keystore for private key)

$forge script script/____.s.sol --rpc-url http://localhost:8545
(script name)
--account ____key --sender public-key --broadcast
(keystore name)
-vvvv
(max. verbosity)

(another way: forge create ____ (--zksync) (--legacy) (contract: 0x00)

cast send contract_address "fn_name(args)" (arg value)
~~--from public-key~~ --keystore path_to_keystore
(send txn (update, write, etc.))

Cast call contract_address "fn_name(arg)" arg_value
(calling fns, (view, read, etc.))

(26/1/25) $foundryup -zksync → switch to zksync ~~foundry~~

$foundryup → revert to base foundry

Endpoint → devices that connects to a network and exchanges data with it.
→ (web3) specific location or address on a blockchain network where a web3 app can interact with and ~~or~~ access ~~defi~~ data.
(e.g. RPC urls)
(address of a unique resource on the internet)

(27/1/25)

Foundry Testing

* Testing a specific part of contract. (unit tests).
* Testing how one part works with ~~other~~ other parts. (Integration)
* Testing the code on a simulated ~~or~~ real environment (forked)
* Testing code in a real non-production environment (staging)

$forge coverage --fork-url $RPC_URL

(28/1/25)

address(0) → default address (zero address)

AAA method for writing tests.

→ Arrange, Act, Assert

chisel

↳ solidity shell

src, .env
scripts, .toml
tests
mock contracts
helper scripts

makeAddr(name)
vm.deal(to, amt)
vm.expectRevert()
vm.prank(address)
hoax(address, value) (vm.prank + vm.deal)

(29/1/25) $forge snapshot → gives how much gas each test fns use.

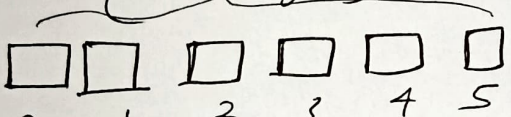
* In anvil, gas price defaults to zero.

vm.txGasPrice(gasprice)
↳ sets a gas price for local chain (anvil)

* storage → permanent data storage location on the blockchain.
→ data persists across txns.

* transient storage → temporary data location.
→ exists only within the current txn and cleared once the txn is completed.

(each 32 bytes)



* value at a certain storage slot —

0x00000 ---- 000
64 zeros } 2 zeros : 1 byte (Hex. digits)

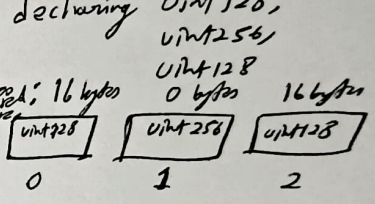
Say, an address var. store some addrss —

address owner = ____;

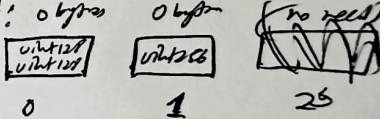
0x0000 — 0 90173c —— 98a1
24 zeros, 40 digits (=20 bytes)
(takes up 20 bytes)

* Data is stored one after the other starting from 1st variable stored in the 1st slot.

* declaring uint128, uint256, uint128



* declaring uint128, uint128, uint256



(storage-efficient)

vm.load(address, bytes32)
→ loads the value found in the provided storage slot of provided address

$forge inspect
contractname storage

$cast storage address slotno

$cast code address

* bytecode → compiled o/p of a contract.
→ low-level machine-readable representation of a contract that can be executed by the EVM. (Ethereum Virtual Machine)
(eg 0x802 == 513)

* opcodes → Human readable low-level instructions that the EVM can interpret.
(operational code)
(eg PUSH1, ADD, CALL)

Yul → intermediate language designed to be compiled into EVM bytecode, offers low-level control and used for optimization.

virtual environment responsible for executing smart contracts securely on the Ethereum blockchain.
part of every Ethereum node.